DEVELOPER GUIDE

AI-Based Monitoring Solution for Oil & Gas Industry

Document Version: 1.0 **Date**: December 2025 **Project Code**: OG-AI-MON-2025 **Audience**: Backend Developers, Frontend Developers, ML Engineers, Data Engineers

---

TABLE OF CONTENTS

1. Development Environment Setup
2. Project Structure
3. Backend Development
4. Frontend Development
5. ML Model Development
6. Data Pipeline Development
7. RAG Development
8. API Reference
9. Testing Guidelines
10. CI/CD Pipeline
11. Coding Standards
12. Troubleshooting

---

1. DEVELOPMENT ENVIRONMENT SETUP

1.1 Prerequisites

Tool	Version	Purpose
Python	3.11+	Backend, ML
Node.js	20 LTS	Frontend
Docker	24+	Containerization
kubectl	1.29+	Kubernetes CLI
Helm	3.13+	K8s package manager
Git	2.40+	Version control
VS Code	Latest	IDE

1.2 Local Development Setup

Step 1: Clone Repositories

```
# Create workspace
mkdir -p ~/oilgas-ai && cd ~/oilgas-ai

# Clone repositories
git clone git@github.com:oilgas/platform-backend.git
git clone git@github.com:oilgas/platform-frontend.git
git clone git@github.com:oilgas/ml-models.git
git clone git@github.com:oilgas/data-pipelines.git
git clone git@github.com:oilgas/rag-service.git
git clone git@github.com:oilgas/infrastructure.git
```

Step 2: Setup Python Environment

```
# Install pyenv
curl https://pyenv.run | bash

# Install Python 3.11
pyenv install 3.11.7
pyenv global 3.11.7

# Create virtual environment
cd platform-backend
python -m venv .venv
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt
pip install -r requirements-dev.txt

# Install pre-commit hooks
pre-commit install
```

Step 3: Setup Node.js Environment

```
# Install nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash

# Install Node.js
nvm install 20
nvm use 20

# Install frontend dependencies
cd ../platform-frontend
npm install
```

Step 4: Setup Docker Environment

```
# Start local services
cd ../infrastructure/docker-compose

# Start databases
docker-compose -f docker-compose.dev.yml up -d postgres influxdb redis milvus kafka

# Verify services
docker-compose ps
```

Step 5: Configure Environment Variables

```
# Copy example env file
cp .env.example .env

# Edit with your local settings
cat > .env << 'EOF'
# Database
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
```

```
POSTGRES_DB=oilgas_dev
POSTGRES_USER=developer
POSTGRES_PASSWORD=devpass123

# InfluxDB
INFLUXDB_URL=http://localhost:8086
INFLUXDB_TOKEN=dev-token-12345
INFLUXDB_ORG=oilgas-dev
INFLUXDB_BUCKET=sensor_data

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379

# Kafka
KAFKA_BOOTSTRAP_SERVERS=localhost:9092

# Milvus
MILVUS_HOST=localhost
MILVUS_PORT=19530

# JWT
JWT_SECRET=dev-secret-key-change-in-production
JWT_ALGORITHM=HS256

# ML
MODEL_REGISTRY_PATH=./models
MLFLOW_TRACKING_URI=http://localhost:5000

# RAG
EMBEDDING_MODEL=BAAI/bge-base-en-v1.5
LLM_MODEL=meta-llama/Llama-2-7b-chat-hf
EOF

# Load environment
source .env
```

Step 6: Initialize Database

```
# Run migrations
cd platform-backend
alembic upgrade head

# Seed test data
python scripts/seed_data.py
```

Step 7: Verify Setup

```
# Run backend tests
pytest tests/ -v

# Run frontend tests
cd ../platform-frontend
npm test
```

```
# Start development servers
# Terminal 1: Backend
cd platform-backend && uvicorn app.main:app --reload --port 8000

# Terminal 2: Frontend
cd platform-frontend && npm run dev

# Access:
# - Backend API: http://localhost:8000/docs
# - Frontend: http://localhost:3000
```

1.3 VS Code Extensions

```
// .vscode/extensions.json
{
  "recommendations": [
    "ms-python.python",
    "ms-python.vscode-pylance",
    "charliermarsh.ruff",
    "ms-python.black-formatter",
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "bradlc.vscode-tailwindcss",
    "ms-azuretools.vscode-docker",
    "ms-kubernetes-tools.vscode-kubernetes-tools",
    "redhat.vscode-yaml",
    "github.copilot"
  ]
}
```

1.4 VS Code Settings

```
// .vscode/settings.json
{
  "python.defaultInterpreterPath": ".venv/bin/python",
  "python.formatting.provider": "none",
  "[python]": {
    "editor.defaultFormatter": "ms-python.black-formatter",
    "editor.formatOnSave": true,
    "editor.codeActionsOnSave": {
      "source.organizeImports": true
    }
  },
  "ruff.lint.args": ["--config=pyproject.toml"],
  "[typescript]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode",
    "editor.formatOnSave": true
  },
  "[typescriptreact]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode",
    "editor.formatOnSave": true
  }
}
```

2. PROJECT STRUCTURE

2.1 Backend Repository Structure

```
platform-backend/
  app/
      __init__.py
      main.py                # FastAPI application entry
      config.py              # Configuration management
      dependencies.py        # Dependency injection

      api/                   # API routes
          __init__.py
          v1/
              __init__.py
              assets.py       # Asset CRUD endpoints
              sensors.py      # Sensor endpoints
              alerts.py       # Alert management
              predictions.py  # ML predictions
              rag.py          # RAG queries
              auth.py         # Authentication
          router.py           # Route aggregation

      models/                # SQLAlchemy models
          __init__.py
          asset.py
          sensor.py
          alert.py
          prediction.py
          user.py

      schemas/               # Pydantic schemas
          __init__.py
          asset.py
          sensor.py
          alert.py
          prediction.py

      services/              # Business logic
          __init__.py
          asset_service.py
          alert_service.py
          prediction_service.py
          rag_service.py
          notification_service.py

      repositories/          # Data access layer
          __init__.py
          base.py
          asset_repository.py
          sensor_repository.py
          alert_repository.py

      core/                  # Core utilities
```

```
        __init__.py
        security.py          # JWT, hashing
        exceptions.py        # Custom exceptions
        logging.py           # Logging config

    integrations/            # External integrations
        __init__.py
        influxdb.py
        kafka.py
        milvus.py
        sap.py
        keycloak.py

alembic/                     # Database migrations
    versions/
    env.py

tests/                       # Test suite
    __init__.py
    conftest.py              # Pytest fixtures
    unit/
    integration/
    e2e/

scripts/                     # Utility scripts
    seed_data.py
    migrate.py

Dockerfile
docker-compose.yml
pyproject.toml               # Project config, dependencies
alembic.ini
requirements.txt
requirements-dev.txt
README.md
```

2.2 Frontend Repository Structure

```
platform-frontend/
  src/
    app/                     # Next.js app router
        layout.tsx
        page.tsx
        dashboard/
            page.tsx
            layout.tsx
        assets/
            page.tsx
            [id]/page.tsx
        alerts/
        predictions/
        chat/                # RAG interface

    components/              # React components
```

```
        ui/                 # Base UI components
            button.tsx
            card.tsx
            table.tsx
            chart.tsx
        assets/
            AssetCard.tsx
            AssetTable.tsx
            AssetHealth.tsx
        alerts/
            AlertList.tsx
            AlertBadge.tsx
        charts/
            TimeSeriesChart.tsx
            GaugeChart.tsx
            HeatMap.tsx
        chat/
             ChatInterface.tsx
             MessageBubble.tsx

    hooks/                  # Custom React hooks
        useAssets.ts
        useSensorData.ts
        useAlerts.ts
        useWebSocket.ts

    lib/                    # Utilities
        api.ts              # API client
        auth.ts             # Authentication
        utils.ts
        constants.ts

    stores/                 # State management (Zustand)
        assetStore.ts
        alertStore.ts
        userStore.ts

    types/                  # TypeScript types
        asset.ts
        sensor.ts
        alert.ts
        api.ts

    styles/
         globals.css

public/
    assets/

tests/
    components/
    e2e/

package.json
```

```
  tsconfig.json
  tailwind.config.js
  next.config.js
  Dockerfile
  README.md
```

2.3 ML Models Repository Structure

```
ml-models/
  anomaly_detection/
      src/
          __init__.py
          model.py            # Model architecture
          train.py            # Training script
          predict.py          # Inference
          preprocess.py       # Data preprocessing
          evaluate.py         # Evaluation metrics
      notebooks/
          01_eda.ipynb
          02_feature_engineering.ipynb
          03_model_training.ipynb
      configs/
          model_config.yaml
          training_config.yaml
      tests/
      README.md

  rul_prediction/
      src/
          model.py            # CNN-LSTM architecture
          train.py
          predict.py
          features.py
      notebooks/
      configs/
      tests/

  computer_vision/
      ppe_detection/
          src/
          data/
          configs/
      corrosion_detection/
      gas_leak_detection/

  shared/
      utils/
      metrics/
      visualization/

  mlflow/
      MLproject

  Dockerfile
```

```
requirements.txt
README.md
```

---

3. BACKEND DEVELOPMENT

3.1 Creating a New API Endpoint

Step 1: Define Pydantic Schema

```python
# app/schemas/asset.py
from pydantic import BaseModel, Field
from typing import Optional, List
from datetime import datetime
from uuid import UUID
from enum import Enum

class AssetType(str, Enum):
    PUMP = "pump"
    COMPRESSOR = "compressor"
    MOTOR = "motor"
    VALVE = "valve"

class AssetStatus(str, Enum):
    OPERATIONAL = "operational"
    DEGRADED = "degraded"
    FAILED = "failed"
    MAINTENANCE = "maintenance"

class AssetBase(BaseModel):
    asset_tag: str = Field(..., min_length=1, max_length=50)
    name: str = Field(..., min_length=1, max_length=255)
    asset_type: AssetType
    location_id: UUID
    criticality: str = "medium"

class AssetCreate(AssetBase):
    """Schema for creating a new asset"""
    specifications: Optional[dict] = None

class AssetUpdate(BaseModel):
    """Schema for updating an asset"""
    name: Optional[str] = None
    status: Optional[AssetStatus] = None
    specifications: Optional[dict] = None

class AssetResponse(AssetBase):
    """Schema for asset response"""
    asset_id: UUID
    status: AssetStatus
    created_at: datetime
    updated_at: datetime
    health_index: Optional[float] = None
```

```
    class Config:
        from_attributes = True

class AssetListResponse(BaseModel):
    """Schema for paginated asset list"""
    data: List[AssetResponse]
    total: int
    page: int
    limit: int
    pages: int
```

Step 2: Create SQLAlchemy Model

```
# app/models/asset.py
from sqlalchemy import Column, String, DateTime, ForeignKey, Enum, JSON
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
import uuid

from app.models.base import Base

class Asset(Base):
    __tablename__ = "assets"
    __table_args__ = {"schema": "asset_mgmt"}

    asset_id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    asset_tag = Column(String(50), unique=True, nullable=False, index=True)
    name = Column(String(255), nullable=False)
    asset_type = Column(String(50), nullable=False, index=True)
    location_id = Column(UUID(as_uuid=True), ForeignKey("asset_mgmt.locations.location_id"))
    parent_asset_id = Column(UUID(as_uuid=True), ForeignKey("asset_mgmt.assets.asset_id"))
    criticality = Column(String(20), default="medium")
    status = Column(String(20), default="operational", index=True)
    specifications = Column(JSON)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(), onupdate=func.now())

    # Relationships
    location = relationship("Location", back_populates="assets")
    sensors = relationship("Sensor", back_populates="asset")
    alerts = relationship("Alert", back_populates="asset")
    parent = relationship("Asset", remote_side=[asset_id], back_populates="children")
    children = relationship("Asset", back_populates="parent")
```

Step 3: Create Repository

```
# app/repositories/asset_repository.py
from typing import List, Optional
from uuid import UUID
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, func
from sqlalchemy.orm import selectinload

from app.models.asset import Asset
```

```python
from app.schemas.asset import AssetCreate, AssetUpdate
from app.repositories.base import BaseRepository

class AssetRepository(BaseRepository[Asset, AssetCreate, AssetUpdate]):
    def __init__(self, session: AsyncSession):
        super().__init__(Asset, session)

    async def get_by_tag(self, asset_tag: str) -> Optional[Asset]:
        """Get asset by tag"""
        stmt = select(Asset).where(Asset.asset_tag == asset_tag)
        result = await self.session.execute(stmt)
        return result.scalar_one_or_none()

    async def get_by_location(
        self,
        location_id: UUID,
        skip: int = 0,
        limit: int = 50
    ) -> List[Asset]:
        """Get assets by location with pagination"""
        stmt = (
            select(Asset)
            .where(Asset.location_id == location_id)
            .options(selectinload(Asset.sensors))
            .offset(skip)
            .limit(limit)
        )
        result = await self.session.execute(stmt)
        return result.scalars().all()

    async def get_by_type_and_status(
        self,
        asset_type: Optional[str] = None,
        status: Optional[str] = None,
        skip: int = 0,
        limit: int = 50
    ) -> tuple[List[Asset], int]:
        """Get assets with filters and total count"""
        stmt = select(Asset)

        if asset_type:
            stmt = stmt.where(Asset.asset_type == asset_type)
        if status:
            stmt = stmt.where(Asset.status == status)

        # Get total count
        count_stmt = select(func.count()).select_from(stmt.subquery())
        total = await self.session.scalar(count_stmt)

        # Get paginated results
        stmt = stmt.offset(skip).limit(limit)
        result = await self.session.execute(stmt)

        return result.scalars().all(), total
```

```python
    async def update_status(self, asset_id: UUID, status: str) -> Optional[Asset]:
        """Update asset status"""
        asset = await self.get(asset_id)
        if asset:
            asset.status = status
            await self.session.commit()
            await self.session.refresh(asset)
        return asset
```

Step 4: Create Service

```python
# app/services/asset_service.py
from typing import List, Optional
from uuid import UUID
from fastapi import HTTPException, status

from app.repositories.asset_repository import AssetRepository
from app.schemas.asset import AssetCreate, AssetUpdate, AssetResponse, AssetListResponse
from app.integrations.influxdb import InfluxDBClient

class AssetService:
    def __init__(
        self,
        repository: AssetRepository,
        influx_client: InfluxDBClient
    ):
        self.repository = repository
        self.influx_client = influx_client

    async def create_asset(self, data: AssetCreate) -> AssetResponse:
        """Create a new asset"""
        # Check if tag already exists
        existing = await self.repository.get_by_tag(data.asset_tag)
        if existing:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail=f"Asset with tag '{data.asset_tag}' already exists"
            )

        asset = await self.repository.create(data)
        return AssetResponse.model_validate(asset)

    async def get_asset(self, asset_id: UUID) -> AssetResponse:
        """Get asset by ID with health index"""
        asset = await self.repository.get(asset_id)
        if not asset:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"Asset {asset_id} not found"
            )

        response = AssetResponse.model_validate(asset)
```

```
    # Get latest health index from InfluxDB
    health_index = await self._get_health_index(asset_id)
    response.health_index = health_index

    return response

async def list_assets(
    self,
    asset_type: Optional[str] = None,
    status: Optional[str] = None,
    page: int = 1,
    limit: int = 50
) -> AssetListResponse:
    """List assets with pagination and filters"""
    skip = (page - 1) * limit
    assets, total = await self.repository.get_by_type_and_status(
        asset_type=asset_type,
        status=status,
        skip=skip,
        limit=limit
    )

    return AssetListResponse(
        data=[AssetResponse.model_validate(a) for a in assets],
        total=total,
        page=page,
        limit=limit,
        pages=(total + limit - 1) // limit
    )

async def update_asset(self, asset_id: UUID, data: AssetUpdate) -> AssetResponse:
    """Update asset"""
    asset = await self.repository.update(asset_id, data)
    if not asset:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Asset {asset_id} not found"
        )
    return AssetResponse.model_validate(asset)

async def delete_asset(self, asset_id: UUID) -> None:
    """Delete asset"""
    deleted = await self.repository.delete(asset_id)
    if not deleted:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Asset {asset_id} not found"
        )

async def _get_health_index(self, asset_id: UUID) -> Optional[float]:
    """Get latest health index from InfluxDB"""
    query = f'''
        from(bucket: "predictions")
        |> range(start: -1h)
```

```
            |> filter(fn: (r) => r._measurement == "predictions")
            |> filter(fn: (r) => r.asset_id == "{asset_id}")
            |> filter(fn: (r) => r._field == "health_index")
            |> last()
        '''
        result = await self.influx_client.query(query)
        if result:
            return result[0].get("_value")
        return None
```

Step 5: Create API Route

```
# app/api/v1/assets.py
from typing import Optional
from uuid import UUID
from fastapi import APIRouter, Depends, Query, status

from app.schemas.asset import (
    AssetCreate, AssetUpdate, AssetResponse, AssetListResponse
)
from app.services.asset_service import AssetService
from app.dependencies import get_asset_service, get_current_user

router = APIRouter(prefix="/assets", tags=["Assets"])

@router.get("", response_model=AssetListResponse)
async def list_assets(
    asset_type: Optional[str] = Query(None, description="Filter by asset type"),
    status: Optional[str] = Query(None, description="Filter by status"),
    page: int = Query(1, ge=1, description="Page number"),
    limit: int = Query(50, ge=1, le=100, description="Items per page"),
    service: AssetService = Depends(get_asset_service),
    current_user = Depends(get_current_user)
):
    """
    List all assets with optional filters.

    - **asset_type**: Filter by pump, compressor, motor, valve
    - **status**: Filter by operational, degraded, failed, maintenance
    - **page**: Page number (default: 1)
    - **limit**: Items per page (default: 50, max: 100)
    """
    return await service.list_assets(
        asset_type=asset_type,
        status=status,
        page=page,
        limit=limit
    )

@router.post("", response_model=AssetResponse, status_code=status.HTTP_201_CREATED)
async def create_asset(
    data: AssetCreate,
    service: AssetService = Depends(get_asset_service),
    current_user = Depends(get_current_user)
```

```
):
    """Create a new asset."""
    return await service.create_asset(data)

@router.get("/{asset_id}", response_model=AssetResponse)
async def get_asset(
    asset_id: UUID,
    service: AssetService = Depends(get_asset_service),
    current_user = Depends(get_current_user)
):
    """Get asset by ID with current health index."""
    return await service.get_asset(asset_id)

@router.put("/{asset_id}", response_model=AssetResponse)
async def update_asset(
    asset_id: UUID,
    data: AssetUpdate,
    service: AssetService = Depends(get_asset_service),
    current_user = Depends(get_current_user)
):
    """Update an asset."""
    return await service.update_asset(asset_id, data)

@router.delete("/{asset_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_asset(
    asset_id: UUID,
    service: AssetService = Depends(get_asset_service),
    current_user = Depends(get_current_user)
):
    """Delete an asset."""
    await service.delete_asset(asset_id)
```

Step 6: Register Route

```
# app/api/router.py
from fastapi import APIRouter
from app.api.v1 import assets, sensors, alerts, predictions, rag, auth

api_router = APIRouter()

api_router.include_router(auth.router)
api_router.include_router(assets.router)
api_router.include_router(sensors.router)
api_router.include_router(alerts.router)
api_router.include_router(predictions.router)
api_router.include_router(rag.router)
```

3.2 Database Migrations

Create Migration

```
# Generate migration from model changes
alembic revision --autogenerate -m "add_asset_table"

# Review generated migration
```

```
cat alembic/versions/<timestamp>_add_asset_table.py
```

Migration File Structure

```
# alembic/versions/20251210_add_asset_table.py
"""add_asset_table

Revision ID: abc123
Revises: xyz789
Create Date: 2025-12-10 10:00:00

"""
from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

# revision identifiers
revision = 'abc123'
down_revision = 'xyz789'
branch_labels = None
depends_on = None

def upgrade() -> None:
    # Create schema
    op.execute("CREATE SCHEMA IF NOT EXISTS asset_mgmt")

    # Create table
    op.create_table(
        'assets',
        sa.Column('asset_id', postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column('asset_tag', sa.String(50), nullable=False, unique=True),
        sa.Column('name', sa.String(255), nullable=False),
        sa.Column('asset_type', sa.String(50), nullable=False),
        sa.Column('location_id', postgresql.UUID(as_uuid=True)),
        sa.Column('status', sa.String(20), default='operational'),
        sa.Column('specifications', postgresql.JSON),
        sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.func.now()),
        sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.func.now()),
        schema='asset_mgmt'
    )

    # Create indexes
    op.create_index('idx_assets_tag', 'assets', ['asset_tag'], schema='asset_mgmt')
    op.create_index('idx_assets_type', 'assets', ['asset_type'], schema='asset_mgmt')

def downgrade() -> None:
    op.drop_table('assets', schema='asset_mgmt')
```

Run Migration

```
# Apply migrations
alembic upgrade head

# Rollback one step
alembic downgrade -1
```

```
# View migration history
alembic history
```

---

4. FRONTEND DEVELOPMENT

4.1 Creating a New Component

Asset Card Component

```tsx
// src/components/assets/AssetCard.tsx
'use client'

import { Card, CardHeader, CardContent, CardFooter } from '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import { HealthGauge } from '@/components/charts/HealthGauge'
import { Asset, AssetStatus } from '@/types/asset'
import { formatDistanceToNow } from 'date-fns'
import Link from 'next/link'

interface AssetCardProps {
  asset: Asset
  onClick?: () => void
}

const statusColors: Record<AssetStatus, string> = {
  operational: 'bg-green-500',
  degraded: 'bg-yellow-500',
  failed: 'bg-red-500',
  maintenance: 'bg-blue-500',
}

export function AssetCard({ asset, onClick }: AssetCardProps) {
  return (
    <Link href={`/assets/${asset.asset_id}`}>
      <Card
        className="hover:shadow-lg transition-shadow cursor-pointer"
        onClick={onClick}
      >
        <CardHeader className="pb-2">
          <div className="flex justify-between items-start">
            <div>
              <h3 className="font-semibold text-lg">{asset.name}</h3>
              <p className="text-sm text-muted-foreground">{asset.asset_tag}</p>
            </div>
            <Badge className={statusColors[asset.status]}>
              {asset.status}
            </Badge>
          </div>
        </CardHeader>

        <CardContent>
          <div className="flex items-center justify-between">
```

```
            <div className="space-y-1">
              <p className="text-sm">
                <span className="text-muted-foreground">Type:</span>{' '}
                {asset.asset_type}
              </p>
              <p className="text-sm">
                <span className="text-muted-foreground">Criticality:</span>{' '}
                <Badge variant="outline">{asset.criticality}</Badge>
              </p>
            </div>

            {asset.health_index !== null && (
              <HealthGauge value={asset.health_index} size="sm" />
            )}
          </div>
        </CardContent>

        <CardFooter className="pt-2 text-xs text-muted-foreground">
          Updated {formatDistanceToNow(new Date(asset.updated_at))} ago
        </CardFooter>
      </Card>
    </Link>
  )
}
```

4.2 Creating a Custom Hook

```
// src/hooks/useAssets.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { api } from '@/lib/api'
import { Asset, AssetCreate, AssetUpdate, AssetListResponse } from '@/types/asset'

interface UseAssetsOptions {
  assetType?: string
  status?: string
  page?: number
  limit?: number
}

export function useAssets(options: UseAssetsOptions = {}) {
  const { assetType, status, page = 1, limit = 50 } = options

  return useQuery<AssetListResponse>({
    queryKey: ['assets', { assetType, status, page, limit }],
    queryFn: async () => {
      const params = new URLSearchParams()
      if (assetType) params.set('asset_type', assetType)
      if (status) params.set('status', status)
      params.set('page', page.toString())
      params.set('limit', limit.toString())

      const response = await api.get(`/assets?${params}`)
      return response.data
    },
```

```
    staleTime: 30000, // 30 seconds
  })
}

export function useAsset(assetId: string) {
  return useQuery<Asset>({
    queryKey: ['asset', assetId],
    queryFn: async () => {
      const response = await api.get(`/assets/${assetId}`)
      return response.data
    },
    enabled: !!assetId,
  })
}

export function useCreateAsset() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: async (data: AssetCreate) => {
      const response = await api.post('/assets', data)
      return response.data
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['assets'] })
    },
  })
}

export function useUpdateAsset() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: async ({ id, data }: { id: string; data: AssetUpdate }) => {
      const response = await api.put(`/assets/${id}`, data)
      return response.data
    },
    onSuccess: (_, variables) => {
      queryClient.invalidateQueries({ queryKey: ['assets'] })
      queryClient.invalidateQueries({ queryKey: ['asset', variables.id] })
    },
  })
}

export function useDeleteAsset() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: async (assetId: string) => {
      await api.delete(`/assets/${assetId}`)
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['assets'] })
    },
```

```
  })
}
```

4.3 Creating a Page

```tsx
// src/app/assets/page.tsx
'use client'

import { useState } from 'react'
import { useAssets } from '@/hooks/useAssets'
import { AssetCard } from '@/components/assets/AssetCard'
import { AssetFilters } from '@/components/assets/AssetFilters'
import { Pagination } from '@/components/ui/pagination'
import { Skeleton } from '@/components/ui/skeleton'
import { Button } from '@/components/ui/button'
import { Plus } from 'lucide-react'
import Link from 'next/link'

export default function AssetsPage() {
  const [filters, setFilters] = useState({
    assetType: undefined,
    status: undefined,
    page: 1,
    limit: 12,
  })

  const { data, isLoading, error } = useAssets(filters)

  if (error) {
    return (
      <div className="p-8 text-center text-red-500">
        Error loading assets: {error.message}
      </div>
    )
  }

  return (
    <div className="container mx-auto py-8">
      <div className="flex justify-between items-center mb-6">
        <h1 className="text-3xl font-bold">Assets</h1>
        <Link href="/assets/new">
          <Button>
            <Plus className="mr-2 h-4 w-4" />
            Add Asset
          </Button>
        </Link>
      </div>

      <AssetFilters
        filters={filters}
        onChange={(newFilters) => setFilters({ ...filters, ...newFilters, page: 1 })}
      />

      {isLoading ? (
```

```
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 mt-6">
          {[...Array(6)].map((_, i) => (
            <Skeleton key={i} className="h-48" />
          ))}
        </div>
      ) : (
        <>
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 mt-6">
            {data?.data.map((asset) => (
              <AssetCard key={asset.asset_id} asset={asset} />
            ))}
          </div>

          {data && data.pages > 1 && (
            <Pagination
              currentPage={filters.page}
              totalPages={data.pages}
              onPageChange={(page) => setFilters({ ...filters, page })}
              className="mt-8"
            />
          )}
        </>
      )}
    </div>
  )
}
```

4.4 Real-time Updates with WebSocket

```
// src/hooks/useWebSocket.ts
import { useEffect, useRef, useCallback } from 'react'
import { useQueryClient } from '@tanstack/react-query'

interface WebSocketMessage {
  type: 'alert' | 'prediction' | 'sensor_update'
  data: any
}

export function useWebSocket(url: string) {
  const ws = useRef<WebSocket | null>(null)
  const queryClient = useQueryClient()

  const connect = useCallback(() => {
    ws.current = new WebSocket(url)

    ws.current.onopen = () => {
      console.log('WebSocket connected')
    }

    ws.current.onmessage = (event) => {
      const message: WebSocketMessage = JSON.parse(event.data)

      switch (message.type) {
        case 'alert':
```

```
            // Invalidate alerts query to refetch
            queryClient.invalidateQueries({ queryKey: ['alerts'] })
            // Show notification
            showNotification(message.data)
            break

          case 'prediction':
            // Update asset health
            queryClient.invalidateQueries({
              queryKey: ['asset', message.data.asset_id]
            })
            break

          case 'sensor_update':
            // Update sensor data in cache
            queryClient.setQueryData(
              ['sensor', message.data.sensor_id],
              message.data
            )
            break
        }
      }

      ws.current.onclose = () => {
        console.log('WebSocket disconnected, reconnecting...')
        setTimeout(connect, 3000)
      }

      ws.current.onerror = (error) => {
        console.error('WebSocket error:', error)
      }
    }, [url, queryClient])

    useEffect(() => {
      connect()

      return () => {
        ws.current?.close()
      }
    }, [connect])

    const send = useCallback((message: any) => {
      if (ws.current?.readyState === WebSocket.OPEN) {
        ws.current.send(JSON.stringify(message))
      }
    }, [])

    return { send }
  }

  function showNotification(alert: any) {
    if ('Notification' in window && Notification.permission === 'granted') {
      new Notification(`Alert: ${alert.title}`, {
        body: alert.description,
```

```
      icon: '/alert-icon.png'
    })
  }
}
```

---

5. ML MODEL DEVELOPMENT

5.1 Training Pipeline

Anomaly Detection Training

```python
# ml-models/anomaly_detection/src/train.py
"""
Anomaly Detection Training Pipeline

Usage:
    python train.py --config configs/training_config.yaml
"""

import argparse
import yaml
from pathlib import Path
from datetime import datetime

import torch
import torch.nn as nn
from torch.utils.data import DataLoader
import numpy as np
import mlflow
import mlflow.pytorch
from loguru import logger

from model import AnomalyAutoencoder
from preprocess import load_data, create_dataset, normalize_features
from evaluate import evaluate_model, plot_metrics

def train(config: dict):
    """Main training function"""

    # Setup MLflow
    mlflow.set_tracking_uri(config['mlflow']['tracking_uri'])
    mlflow.set_experiment(config['mlflow']['experiment_name'])

    with mlflow.start_run(run_name=f"train_{datetime.now().strftime('%Y%m%d_%H%M')}"):

        # Log config
        mlflow.log_params(config['model'])
        mlflow.log_params(config['training'])

        # Load and preprocess data
        logger.info("Loading training data...")
        train_data, val_data = load_data(
            config['data']['train_path'],
```

```
    config['data']['val_split']
)

# Normalize
train_norm, val_norm, norm_stats = normalize_features(train_data, val_data)
mlflow.log_dict(norm_stats, 'normalization_stats.json')

# Create dataloaders
train_dataset = create_dataset(train_norm)
val_dataset = create_dataset(val_norm)

train_loader = DataLoader(
    train_dataset,
    batch_size=config['training']['batch_size'],
    shuffle=True,
    num_workers=4
)
val_loader = DataLoader(
    val_dataset,
    batch_size=config['training']['batch_size']
)

# Initialize model
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
logger.info(f"Using device: {device}")

model = AnomalyAutoencoder(
    input_dim=config['model']['input_dim'],
    latent_dim=config['model']['latent_dim'],
    hidden_dims=config['model']['hidden_dims']
).to(device)

# Loss and optimizer
criterion = nn.MSELoss()
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=config['training']['learning_rate'],
    weight_decay=config['training']['weight_decay']
)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    patience=config['training']['scheduler_patience'],
    factor=0.5
)

# Training loop
best_val_loss = float('inf')
patience_counter = 0

for epoch in range(config['training']['epochs']):
    # Train
    model.train()
    train_loss = 0
```

```
    for batch in train_loader:
        batch = batch.to(device)
        optimizer.zero_grad()

        reconstructed, _ = model(batch)
        loss = criterion(reconstructed, batch)

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        train_loss += loss.item()

    train_loss /= len(train_loader)

    # Validate
    model.eval()
    val_loss = 0
    reconstruction_errors = []

    with torch.no_grad():
        for batch in val_loader:
            batch = batch.to(device)
            reconstructed, _ = model(batch)
            loss = criterion(reconstructed, batch)
            val_loss += loss.item()

            # Per-sample error
            errors = torch.mean((batch - reconstructed) ** 2, dim=1)
            reconstruction_errors.extend(errors.cpu().numpy())

    val_loss /= len(val_loader)
    scheduler.step(val_loss)

    # Log metrics
    mlflow.log_metrics({
        'train_loss': train_loss,
        'val_loss': val_loss,
        'learning_rate': optimizer.param_groups[0]['lr']
    }, step=epoch)

    logger.info(f"Epoch {epoch+1}: train_loss={train_loss:.6f}, val_loss={val_loss:.6f}")

    # Early stopping
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        patience_counter = 0
        torch.save(model.state_dict(), 'best_model.pt')
    else:
        patience_counter += 1
        if patience_counter >= config['training']['early_stopping_patience']:
            logger.info(f"Early stopping at epoch {epoch+1}")
            break
```

```
        # Calculate threshold from reconstruction errors
        errors = np.array(reconstruction_errors)
        threshold_config = {
            'mean': float(np.mean(errors)),
            'std': float(np.std(errors)),
            'threshold_2sigma': float(np.mean(errors) + 2 * np.std(errors)),
            'threshold_3sigma': float(np.mean(errors) + 3 * np.std(errors))
        }
        mlflow.log_dict(threshold_config, 'threshold_config.json')

        # Load best model and evaluate
        model.load_state_dict(torch.load('best_model.pt'))
        metrics = evaluate_model(model, val_loader, device, threshold_config)
        mlflow.log_metrics(metrics)

        # Plot and log figures
        fig = plot_metrics(reconstruction_errors, threshold_config)
        mlflow.log_figure(fig, 'reconstruction_error_distribution.png')

        # Export to ONNX
        model.eval()
        dummy_input = torch.randn(1, config['model']['input_dim']).to(device)
        torch.onnx.export(
            model,
            dummy_input,
            'model.onnx',
            input_names=['input'],
            output_names=['reconstructed', 'latent'],
            dynamic_axes={'input': {0: 'batch_size'}}
        )

        # Log artifacts
        mlflow.log_artifact('best_model.pt')
        mlflow.log_artifact('model.onnx')

        # Register model
        mlflow.pytorch.log_model(
            model,
            "model",
            registered_model_name=config['mlflow']['model_name']
        )

        logger.info("Training complete!")
        logger.info(f"Best validation loss: {best_val_loss:.6f}")
        logger.info(f"Threshold (2-sigma): {threshold_config['threshold_2sigma']:.6f}")

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--config', required=True, help='Path to config file')
    args = parser.parse_args()

    with open(args.config) as f:
        config = yaml.safe_load(f)
```

```
    train(config)

if __name__ == '__main__':
    main()
```

Training Configuration

```yaml
# ml-models/anomaly_detection/configs/training_config.yaml
mlflow:
  tracking_uri: http://mlflow.oilgas.internal:5000
  experiment_name: anomaly_detection
  model_name: anomaly-detector

data:
  train_path: s3://oilgas-data/training/vibration_features.parquet
  val_split: 0.2

model:
  input_dim: 15
  latent_dim: 3
  hidden_dims: [10, 5]

training:
  batch_size: 256
  epochs: 100
  learning_rate: 0.001
  weight_decay: 0.0001
  scheduler_patience: 5
  early_stopping_patience: 10
```

---

6. DATA PIPELINE DEVELOPMENT

6.1 Kafka Consumer

```python
# data-pipelines/src/consumers/sensor_consumer.py
"""
Sensor Data Consumer

Consumes sensor readings from Kafka and writes to InfluxDB.
"""

import json
from datetime import datetime
from typing import List
import asyncio

from confluent_kafka import Consumer, KafkaError
from influxdb_client import InfluxDBClient, Point
from influxdb_client.client.write_api import ASYNCHRONOUS
from loguru import logger
from pydantic import BaseModel, validator
```

```
class SensorReading(BaseModel):
    """Sensor reading schema"""
    sensor_id: str
    asset_id: str
    timestamp: datetime
    value: float
    quality: int
    unit: str

    @validator('timestamp', pre=True)
    def parse_timestamp(cls, v):
        if isinstance(v, str):
            return datetime.fromisoformat(v.replace('Z', '+00:00'))
        return v

class SensorConsumer:
    def __init__(self, config: dict):
        self.config = config

        # Kafka consumer
        self.consumer = Consumer({
            'bootstrap.servers': config['kafka']['bootstrap_servers'],
            'group.id': config['kafka']['consumer_group'],
            'auto.offset.reset': 'earliest',
            'enable.auto.commit': False,
        })
        self.consumer.subscribe(config['kafka']['topics'])

        # InfluxDB client
        self.influx_client = InfluxDBClient(
            url=config['influxdb']['url'],
            token=config['influxdb']['token'],
            org=config['influxdb']['org']
        )
        self.write_api = self.influx_client.write_api(write_options=ASYNCHRONOUS)
        self.bucket = config['influxdb']['bucket']

        self.batch: List[Point] = []
        self.batch_size = config.get('batch_size', 1000)
        self.flush_interval = config.get('flush_interval_seconds', 5)

    def process_message(self, message: bytes) -> SensorReading:
        """Parse and validate message"""
        data = json.loads(message)
        return SensorReading(**data)

    def to_influx_point(self, reading: SensorReading) -> Point:
        """Convert reading to InfluxDB point"""
        return (
            Point("sensor_readings")
            .tag("sensor_id", reading.sensor_id)
            .tag("asset_id", reading.asset_id)
            .field("value", reading.value)
            .field("quality", reading.quality)
```

```
            .time(reading.timestamp)
        )

    async def flush_batch(self):
        """Write batch to InfluxDB"""
        if not self.batch:
            return

        try:
            self.write_api.write(bucket=self.bucket, record=self.batch)
            logger.info(f"Flushed {len(self.batch)} points to InfluxDB")
            self.batch = []
            self.consumer.commit()
        except Exception as e:
            logger.error(f"Error writing to InfluxDB: {e}")

    async def run(self):
        """Main consumer loop"""
        logger.info("Starting sensor consumer...")

        last_flush = datetime.now()

        try:
            while True:
                msg = self.consumer.poll(1.0)

                if msg is None:
                    # Check if flush needed
                    if (datetime.now() - last_flush).seconds >= self.flush_interval:
                        await self.flush_batch()
                        last_flush = datetime.now()
                    continue

                if msg.error():
                    if msg.error().code() == KafkaError._PARTITION_EOF:
                        continue
                    logger.error(f"Kafka error: {msg.error()}")
                    continue

                try:
                    reading = self.process_message(msg.value())
                    point = self.to_influx_point(reading)
                    self.batch.append(point)

                    if len(self.batch) >= self.batch_size:
                        await self.flush_batch()
                        last_flush = datetime.now()

                except Exception as e:
                    logger.error(f"Error processing message: {e}")

        except KeyboardInterrupt:
            logger.info("Shutting down...")
        finally:
```

```
            await self.flush_batch()
            self.consumer.close()
            self.influx_client.close()

if __name__ == '__main__':
    import yaml

    with open('config.yaml') as f:
        config = yaml.safe_load(f)

    consumer = SensorConsumer(config)
    asyncio.run(consumer.run())
```

6.2 Feature Engineering Pipeline

```
# data-pipelines/src/features/vibration_features.py
"""
Vibration Feature Engineering Pipeline

Computes time-domain and frequency-domain features from raw vibration data.
"""

import numpy as np
from scipy import signal, stats
from scipy.fft import rfft, rfftfreq
from dataclasses import dataclass
from typing import Dict, Any

@dataclass
class VibrationFeatures:
    """Container for vibration features"""
    # Time domain
    rms: float
    peak: float
    peak_to_peak: float
    crest_factor: float
    kurtosis: float
    skewness: float

    # Frequency domain
    fft_1x: float
    fft_2x: float
    fft_3x: float

    # Bearing frequencies
    bpfo: float
    bpfi: float
    bsf: float
    ftf: float

    # Envelope
    envelope_rms: float
    envelope_kurtosis: float
```

```
    def to_dict(self) -> Dict[str, float]:
        return {
            'rms': self.rms,
            'peak': self.peak,
            'peak_to_peak': self.peak_to_peak,
            'crest_factor': self.crest_factor,
            'kurtosis': self.kurtosis,
            'skewness': self.skewness,
            'fft_1x': self.fft_1x,
            'fft_2x': self.fft_2x,
            'fft_3x': self.fft_3x,
            'bpfo': self.bpfo,
            'bpfi': self.bpfi,
            'bsf': self.bsf,
            'ftf': self.ftf,
            'envelope_rms': self.envelope_rms,
            'envelope_kurtosis': self.envelope_kurtosis,
        }

    def to_array(self) -> np.ndarray:
        return np.array(list(self.to_dict().values()))


def extract_features(
    waveform: np.ndarray,
    sampling_rate: float,
    rpm: float,
    bearing_params: Dict[str, float]
) -> VibrationFeatures:
    """
    Extract vibration features from raw waveform.

    Args:
        waveform: Raw vibration signal
        sampling_rate: Sampling rate in Hz
        rpm: Rotational speed in RPM
        bearing_params: Bearing coefficients (bpfo, bpfi, bsf, ftf)

    Returns:
        VibrationFeatures object
    """
    # Remove DC offset
    waveform = waveform - np.mean(waveform)

    # Time domain features
    rms = np.sqrt(np.mean(waveform ** 2))
    peak = np.max(np.abs(waveform))
    peak_to_peak = np.max(waveform) - np.min(waveform)
    crest_factor = peak / rms if rms > 0 else 0
    kurtosis = stats.kurtosis(waveform)
    skewness = stats.skew(waveform)

    # FFT
    n_samples = len(waveform)
```

```
    fft_values = np.abs(rfft(waveform)) / n_samples
    frequencies = rfftfreq(n_samples, 1 / sampling_rate)

    # Rotational frequency
    rot_freq = rpm / 60

    # Get amplitude at specific frequencies
    def get_amplitude(target_freq: float, bandwidth: float = 5) -> float:
        mask = (frequencies >= target_freq - bandwidth) & (frequencies <= target_freq + bandwidth)
        if np.any(mask):
            return float(np.max(fft_values[mask]))
        return 0.0

    fft_1x = get_amplitude(rot_freq)
    fft_2x = get_amplitude(2 * rot_freq)
    fft_3x = get_amplitude(3 * rot_freq)

    # Bearing frequencies
    bpfo = get_amplitude(bearing_params['bpfo'] * rot_freq)
    bpfi = get_amplitude(bearing_params['bpfi'] * rot_freq)
    bsf = get_amplitude(bearing_params['bsf'] * rot_freq)
    ftf = get_amplitude(bearing_params['ftf'] * rot_freq)

    # Envelope analysis
    analytic_signal = signal.hilbert(waveform)
    envelope = np.abs(analytic_signal)
    envelope_rms = np.sqrt(np.mean(envelope ** 2))
    envelope_kurtosis = stats.kurtosis(envelope)

    return VibrationFeatures(
        rms=rms,
        peak=peak,
        peak_to_peak=peak_to_peak,
        crest_factor=crest_factor,
        kurtosis=kurtosis,
        skewness=skewness,
        fft_1x=fft_1x,
        fft_2x=fft_2x,
        fft_3x=fft_3x,
        bpfo=bpfo,
        bpfi=bpfi,
        bsf=bsf,
        ftf=ftf,
        envelope_rms=envelope_rms,
        envelope_kurtosis=envelope_kurtosis,
    )


# Example bearing coefficients for SKF 6205 bearing
SKF_6205_PARAMS = {
    'bpfo': 3.585,  # Ball Pass Frequency Outer
    'bpfi': 5.415,  # Ball Pass Frequency Inner
    'bsf': 2.357,   # Ball Spin Frequency
    'ftf': 0.398,   # Fundamental Train Frequency
```

```
}
```

---

7. RAG DEVELOPMENT

7.1 Document Ingestion

```python
# rag-service/src/ingestion/pipeline.py
"""
Document Ingestion Pipeline

Processes documents, chunks them, generates embeddings, and stores in Milvus.
"""

from pathlib import Path
from typing import List, Generator
import hashlib
from datetime import datetime

from unstructured.partition.auto import partition
from langchain.text_splitter import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer
from pymilvus import Collection, connections
import psycopg2
from loguru import logger

class DocumentIngester:
    def __init__(self, config: dict):
        self.config = config

        # Embedding model
        logger.info(f"Loading embedding model: {config['embedding_model']}")
        self.embedder = SentenceTransformer(config['embedding_model'])

        # Text splitter
        self.splitter = RecursiveCharacterTextSplitter(
            chunk_size=config['chunk_size'],
            chunk_overlap=config['chunk_overlap'],
            separators=["\n\n", "\n", ". ", " ", ""]
        )

        # Milvus
        connections.connect(
            host=config['milvus_host'],
            port=config['milvus_port']
        )
        self.collection = Collection(config['collection_name'])

        # PostgreSQL
        self.pg_conn = psycopg2.connect(config['postgres_dsn'])

    def parse_document(self, file_path: Path) -> dict:
        """Parse document and extract text"""
```

```
    logger.info(f"Parsing document: {file_path.name}")

    elements = partition(str(file_path))
    text_content = "\n\n".join([str(el) for el in elements])

    # Compute checksum
    with open(file_path, 'rb') as f:
        checksum = hashlib.sha256(f.read()).hexdigest()

    return {
        'content': text_content,
        'file_name': file_path.name,
        'file_type': file_path.suffix[1:],
        'file_size': file_path.stat().st_size,
        'checksum': checksum
    }

def chunk_document(self, content: str) -> List[str]:
    """Split document into chunks"""
    return self.splitter.split_text(content)

def embed_chunks(self, chunks: List[str]) -> List[List[float]]:
    """Generate embeddings for chunks"""
    return self.embedder.encode(
        chunks,
        normalize_embeddings=True,
        show_progress_bar=True
    ).tolist()

def store_document(self, doc_id: str, parsed: dict, chunks: List[str], embeddings: List[List[float]]
    """Store document metadata and chunks"""

    # Store document metadata in PostgreSQL
    with self.pg_conn.cursor() as cur:
        cur.execute("""
            INSERT INTO knowledge.documents
            (document_id, file_name, file_type, file_size_bytes, checksum_sha256, indexed_at, status
            VALUES (%s, %s, %s, %s, %s, %s, 'indexed')
            ON CONFLICT (document_id) DO UPDATE SET indexed_at = EXCLUDED.indexed_at
        """, (doc_id, parsed['file_name'], parsed['file_type'],
              parsed['file_size'], parsed['checksum'], datetime.utcnow()))

        # Store chunks
        for i, chunk in enumerate(chunks):
            chunk_id = f"{doc_id}_{i:04d}"
            cur.execute("""
                INSERT INTO knowledge.chunks
                (chunk_id, document_id, chunk_index, content, token_count)
                VALUES (%s, %s, %s, %s, %s)
                ON CONFLICT DO NOTHING
            """, (chunk_id, doc_id, i, chunk, len(chunk.split())))

    self.pg_conn.commit()
```

```
        # Store embeddings in Milvus
        chunk_ids = [f"{doc_id}_{i:04d}" for i in range(len(chunks))]
        doc_ids = [doc_id] * len(chunks)
        previews = [c[:500] for c in chunks]

        self.collection.insert([chunk_ids, doc_ids, embeddings, previews])
        self.collection.flush()

        logger.info(f"Stored {len(chunks)} chunks for document {doc_id}")

    def ingest_document(self, file_path: Path) -> dict:
        """Full ingestion pipeline for single document"""
        doc_id = hashlib.md5(str(file_path).encode()).hexdigest()

        # Parse
        parsed = self.parse_document(file_path)

        # Chunk
        chunks = self.chunk_document(parsed['content'])
        logger.info(f"Created {len(chunks)} chunks")

        # Embed
        embeddings = self.embed_chunks(chunks)

        # Store
        self.store_document(doc_id, parsed, chunks, embeddings)

        return {
            'document_id': doc_id,
            'file_name': parsed['file_name'],
            'chunk_count': len(chunks)
        }

    def ingest_directory(self, directory: Path) -> Generator:
        """Ingest all documents in directory"""
        supported = {'.pdf', '.docx', '.doc', '.txt', '.html', '.md'}

        files = [f for f in directory.rglob('*') if f.is_file() and f.suffix.lower() in supported]
        logger.info(f"Found {len(files)} documents to ingest")

        for file_path in files:
            try:
                result = self.ingest_document(file_path)
                yield {'status': 'success', **result}
            except Exception as e:
                logger.error(f"Error ingesting {file_path}: {e}")
                yield {'status': 'error', 'file_name': file_path.name, 'error': str(e)}
```

7.2 RAG Query Pipeline

```
# rag-service/src/query/pipeline.py
"""
RAG Query Pipeline
```

```
Handles query processing, retrieval, reranking, and generation.
"""

from typing import List, Dict, Optional
import hashlib
import json
from datetime import datetime

from sentence_transformers import SentenceTransformer, CrossEncoder
from pymilvus import Collection, connections
from vllm import LLM, SamplingParams
import redis
from loguru import logger

class RAGPipeline:
    def __init__(self, config: dict):
        self.config = config

        # Embedding model
        self.embedder = SentenceTransformer(config['embedding_model'])

        # Reranker
        self.reranker = CrossEncoder(config['reranker_model'])

        # Milvus
        connections.connect(host=config['milvus_host'], port=config['milvus_port'])
        self.collection = Collection(config['collection_name'])
        self.collection.load()

        # LLM
        self.llm = LLM(
            model=config['llm_model'],
            tensor_parallel_size=config['tensor_parallel_size']
        )
        self.sampling_params = SamplingParams(
            temperature=config['temperature'],
            max_tokens=config['max_tokens']
        )

        # Cache
        self.cache = redis.Redis(
            host=config['redis_host'],
            port=config['redis_port']
        )
        self.cache_ttl = config['cache_ttl']

        # Prompt template
        self.system_prompt = """You are an expert Oil & Gas engineer assistant.
Answer questions based ONLY on the provided context.
If the context doesn't contain enough information, say so clearly.
Always cite sources using [Source: document_name] format.

Context:
{context}
```

```
Current Equipment Status (if provided):
{equipment_context}

Question: {question}

Answer:"""

    def get_cache_key(self, query: str) -> str:
        return f"rag:{hashlib.sha256(query.encode()).hexdigest()}"

    def embed_query(self, query: str) -> List[float]:
        return self.embedder.encode(query, normalize_embeddings=True).tolist()

    def vector_search(self, embedding: List[float], top_k: int = 20) -> List[Dict]:
        """Search Milvus for relevant chunks"""
        results = self.collection.search(
            data=[embedding],
            anns_field="embedding",
            param={"metric_type": "COSINE", "params": {"nprobe": 10}},
            limit=top_k,
            output_fields=["chunk_id", "document_id", "content_preview"]
        )

        chunks = []
        for hit in results[0]:
            chunks.append({
                'chunk_id': hit.entity.get('chunk_id'),
                'document_id': hit.entity.get('document_id'),
                'content': hit.entity.get('content_preview'),
                'score': hit.score
            })
        return chunks

    def rerank(self, query: str, chunks: List[Dict], top_k: int = 5) -> List[Dict]:
        """Rerank chunks using cross-encoder"""
        if not chunks:
            return []

        pairs = [(query, c['content']) for c in chunks]
        scores = self.reranker.predict(pairs)

        for i, score in enumerate(scores):
            chunks[i]['rerank_score'] = float(score)

        return sorted(chunks, key=lambda x: x['rerank_score'], reverse=True)[:top_k]

    def format_context(self, chunks: List[Dict]) -> str:
        """Format chunks for prompt"""
        context_parts = []
        for i, chunk in enumerate(chunks, 1):
            context_parts.append(f"[Document {i}]\n{chunk['content']}\n")
        return "\n---\n".join(context_parts)
```

```
def generate(self, query: str, context: str, equipment_context: str = "") -> str:
    """Generate response using LLM"""
    prompt = self.system_prompt.format(
        context=context,
        equipment_context=equipment_context or "No equipment context provided.",
        question=query
    )

    outputs = self.llm.generate([prompt], self.sampling_params)
    return outputs[0].outputs[0].text

def query(
    self,
    query: str,
    equipment_context: Optional[Dict] = None,
    use_cache: bool = True
) -> Dict:
    """Execute full RAG pipeline"""
    start_time = datetime.now()

    # Check cache
    cache_key = self.get_cache_key(query)
    if use_cache:
        cached = self.cache.get(cache_key)
        if cached:
            result = json.loads(cached)
            result['cached'] = True
            return result

    # Embed query
    embedding = self.embed_query(query)

    # Search
    chunks = self.vector_search(embedding, top_k=20)

    # Rerank
    reranked = self.rerank(query, chunks, top_k=5)

    # Format context
    context = self.format_context(reranked)
    equip_ctx = json.dumps(equipment_context, indent=2) if equipment_context else ""

    # Generate
    response = self.generate(query, context, equip_ctx)

    # Prepare result
    latency_ms = int((datetime.now() - start_time).total_seconds() * 1000)

    result = {
        'query': query,
        'response': response,
        'sources': [
            {
                'document_id': c['document_id'],
```

```
                'chunk_preview': c['content'][:200],
                'score': c['rerank_score']
            }
            for c in reranked
        ],
        'latency_ms': latency_ms,
        'cached': False
    }

    # Cache result
    if use_cache:
        self.cache.setex(cache_key, self.cache_ttl, json.dumps(result))

    return result
```

---

8. API REFERENCE

8.1 Authentication

```
# Get access token
curl -X POST https://api.oilgas.internal/v1/auth/token \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "username=user@oilgas.com&password=secret"

# Response
{
  "access_token": "eyJhbG...",
  "token_type": "bearer",
  "expires_in": 3600
}

# Use token in requests
curl https://api.oilgas.internal/v1/assets \
  -H "Authorization: Bearer eyJhbG..."
```

8.2 Assets API

Method	Endpoint	Description
GET	/v1/assets	List assets
POST	/v1/assets	Create asset
GET	/v1/assets/{id}	Get asset
PUT	/v1/assets/{id}	Update asset
DELETE	/v1/assets/{id}	Delete asset
GET	/v1/assets/{id}/health	Get asset health
GET	/v1/assets/{id}/sensors	Get asset sensors
GET	/v1/assets/{id}/predictions	Get predictions

8.3 RAG API

Method	Endpoint	Description
POST	/v1/rag/query	Query knowledge base
POST	/v1/rag/feedback	Submit feedback
GET	/v1/rag/documents	List documents
POST	/v1/rag/documents	Upload document

---

9. TESTING GUIDELINES

9.1 Unit Tests

```python
# tests/unit/test_asset_service.py
import pytest
from unittest.mock import AsyncMock, MagicMock
from uuid import uuid4

from app.services.asset_service import AssetService
from app.schemas.asset import AssetCreate

@pytest.fixture
def mock_repository():
    return AsyncMock()

@pytest.fixture
def mock_influx():
    return AsyncMock()

@pytest.fixture
def asset_service(mock_repository, mock_influx):
    return AssetService(mock_repository, mock_influx)

@pytest.mark.asyncio
async def test_create_asset_success(asset_service, mock_repository):
    # Arrange
    data = AssetCreate(
        asset_tag="PMP-001",
        name="Test Pump",
        asset_type="pump",
        location_id=uuid4()
    )
    mock_repository.get_by_tag.return_value = None
    mock_repository.create.return_value = MagicMock(
        asset_id=uuid4(),
        asset_tag="PMP-001",
        name="Test Pump",
        asset_type="pump",
        status="operational"
    )

    # Act
    result = await asset_service.create_asset(data)
```

```
    # Assert
    assert result.asset_tag == "PMP-001"
    mock_repository.create.assert_called_once()

@pytest.mark.asyncio
async def test_create_asset_duplicate_tag(asset_service, mock_repository):
    # Arrange
    data = AssetCreate(
        asset_tag="PMP-001",
        name="Test Pump",
        asset_type="pump",
        location_id=uuid4()
    )
    mock_repository.get_by_tag.return_value = MagicMock()  # Existing asset

    # Act & Assert
    with pytest.raises(HTTPException) as exc:
        await asset_service.create_asset(data)

    assert exc.value.status_code == 409
```

9.2 Integration Tests

```
# tests/integration/test_assets_api.py
import pytest
from httpx import AsyncClient
from app.main import app

@pytest.fixture
async def client():
    async with AsyncClient(app=app, base_url="http://test") as ac:
        yield ac

@pytest.fixture
async def auth_headers(client):
    response = await client.post("/v1/auth/token", data={
        "username": "test@oilgas.com",
        "password": "testpass"
    })
    token = response.json()["access_token"]
    return {"Authorization": f"Bearer {token}"}

@pytest.mark.asyncio
async def test_create_and_get_asset(client, auth_headers):
    # Create asset
    create_response = await client.post(
        "/v1/assets",
        json={
            "asset_tag": "TEST-001",
            "name": "Test Asset",
            "asset_type": "pump",
            "location_id": "550e8400-e29b-41d4-a716-446655440000"
        },
        headers=auth_headers
```

```
    )
    assert create_response.status_code == 201
    asset_id = create_response.json()["asset_id"]

    # Get asset
    get_response = await client.get(
        f"/v1/assets/{asset_id}",
        headers=auth_headers
    )
    assert get_response.status_code == 200
    assert get_response.json()["asset_tag"] == "TEST-001"
```

9.3 Running Tests

```
# Run all tests
pytest tests/ -v

# Run with coverage
pytest tests/ --cov=app --cov-report=html

# Run specific test file
pytest tests/unit/test_asset_service.py -v

# Run tests matching pattern
pytest tests/ -k "test_create" -v
```

---

10. CI/CD PIPELINE

10.1 GitHub Actions

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: testpass
        ports:
          - 5432:5432

    steps:
      - uses: actions/checkout@v4
```

```yaml
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install -r requirements-dev.txt

      - name: Lint
        run: |
          ruff check .
          black --check .

      - name: Test
        run: |
          pytest tests/ --cov=app --cov-report=xml

      - name: Upload coverage
        uses: codecov/codecov-action@v3

  build:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - uses: actions/checkout@v4

      - name: Build Docker image
        run: |
          docker build -t oilgas/api:${{ github.sha }} .

      - name: Push to registry
        run: |
          echo ${{ secrets.DOCKER_PASSWORD }} | docker login -u ${{ secrets.DOCKER_USERNAME }} --passwo
          docker push oilgas/api:${{ github.sha }}
```

---

11. CODING STANDARDS

11.1 Python Style Guide

- Follow PEP 8
- Use Black for formatting (line length 100)
- Use Ruff for linting
- Type hints required for all functions
- Docstrings required for public functions

```python
# Good example
def calculate_health_index(
    sensor_readings: List[SensorReading],
```

```
    weights: Dict[str, float]
) -> float:
    """
    Calculate asset health index from sensor readings.

    Args:
        sensor_readings: List of recent sensor readings
        weights: Weight for each sensor type

    Returns:
        Health index between 0 (failed) and 1 (healthy)

    Raises:
        ValueError: If sensor_readings is empty
    """
    if not sensor_readings:
        raise ValueError("sensor_readings cannot be empty")

    weighted_sum = sum(
        reading.normalized_value * weights.get(reading.sensor_type, 1.0)
        for reading in sensor_readings
    )
    total_weight = sum(weights.values())

    return weighted_sum / total_weight
```

11.2 TypeScript Style Guide

- Use ESLint + Prettier
- Strict TypeScript mode
- Functional components with hooks
- Named exports preferred

```typescript
// Good example
interface AssetCardProps {
  asset: Asset
  onSelect?: (asset: Asset) => void
}

export function AssetCard({ asset, onSelect }: AssetCardProps) {
  const handleClick = () => {
    onSelect?.(asset)
  }

  return (
    <Card onClick={handleClick}>
      <CardHeader>
        <h3>{asset.name}</h3>
      </CardHeader>
    </Card>
  )
}
```

11.3 Git Commit Messages

```
<type>(<scope>): <subject>

<body>

<footer>

Types: feat, fix, docs, style, refactor, test, chore
Scope: api, frontend, ml, data, infra

Examples:
feat(api): add asset health endpoint
fix(frontend): resolve chart rendering issue
docs(api): update OpenAPI specification
test(ml): add unit tests for anomaly detector
```

---

12. TROUBLESHOOTING

12.1 Common Issues

Database Connection Error

```python
# Check connection
from app.config import settings
import psycopg2

try:
    conn = psycopg2.connect(settings.database_url)
    print("Connection successful")
except Exception as e:
    print(f"Connection failed: {e}")

# Solutions:
# 1. Check DATABASE_URL environment variable
# 2. Verify PostgreSQL is running
# 3. Check network connectivity
# 4. Verify credentials
```

Model Loading Error

```python
# Debug model loading
import torch

# Check CUDA availability
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"CUDA device count: {torch.cuda.device_count()}")

# Check model file
from pathlib import Path
model_path = Path("models/anomaly_detector.pt")
print(f"Model exists: {model_path.exists()}")
print(f"Model size: {model_path.stat().st_size / 1e6:.2f} MB")
```

Kafka Consumer Lag

```
# Check consumer lag
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group ai-platform

# Solutions:
# 1. Scale up consumers
# 2. Increase batch size
# 3. Optimize processing logic
# 4. Check for slow external calls
```

END OF DEVELOPER GUIDE